

Number Systems

By: Tanmoy Biswas,

Department of Computer Science,

Syamaprasad College



Objective

- Weighted and non-weighted coding systems
- Positional numbering systems such as binary, octal, and hexadecimal.
- 8-4-2-1
- 2-4-2-1
- Binary Coded Decimal (BCD).
- Gray codes.

Weighted

Weighted codes are a system of representing numbers where each digit of a number is assigned a **specific weight based on its position**. In these codes, the value of a digit is determined by multiplying the digit by the weight associated with its position and then summing the results for all positions.

Example; Binary number system

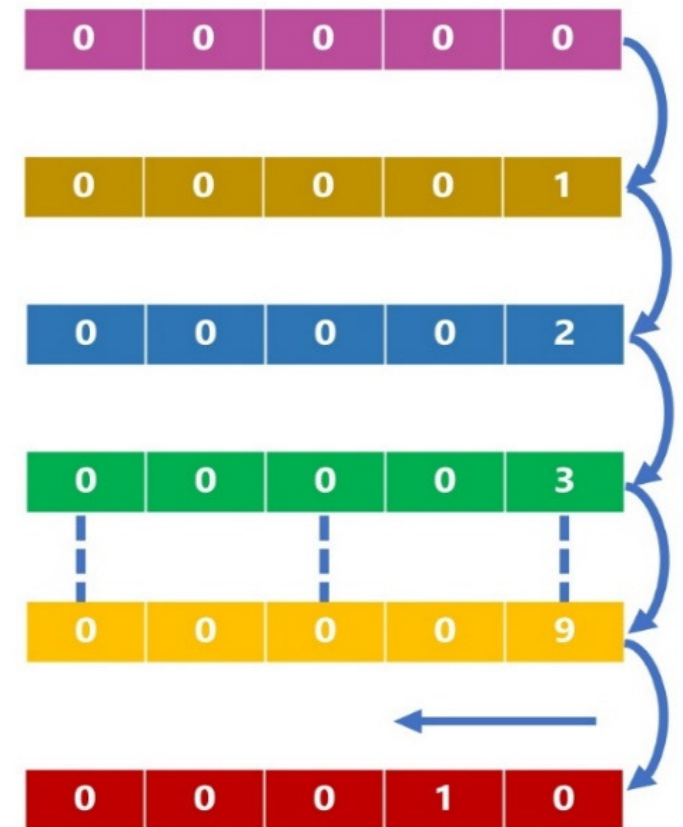
	Positional Weights					
M.S.B →	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰	← L.S.B

[illegible]

Decimal number system

To understand how the decimal number system works we need to check the decimal odometer as found in the cars and other vehicles as shown below;

- Suppose it is reset to **00000**
- After 1 km the reading becomes **00001**
- Successive kms produce **00002**, **00003**, and so on, up to **00009** and so on.
- A familiar thing happens at the end of the tenth km. When the units wheel turns from 9 back to 0, a tab on this wheel forces the tens wheel to advance by 1. This is why the numbers change to 00010.



Decimal number system

To understand how the decimal number system works we need to check the positional weights given below;

4486.265



10^{-4}	10^{-3}	10^{-2}	10^{-1}	10^0	10^1	10^2	10^3
-	5	6	2	6	8	4	4
Fractional Part				Unit	Ten	Hundred	Thousand

Decimal number **4486.265**, the integer part (i.e. 4486) can be expressed as;

$$\mathbf{4486} = 6 \times 10^0 + 8 \times 10^1 + 4 \times 10^2 + 4 \times 10^3 = 6 + 80 + 500 + 3000 = 3586$$

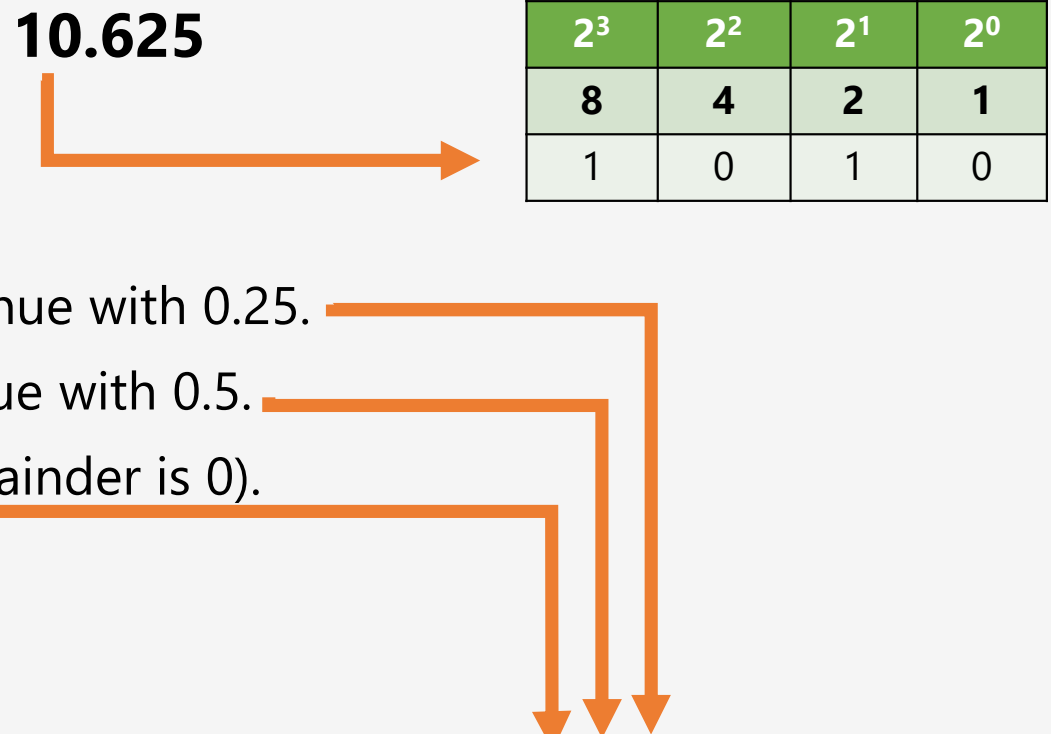
and the fractional part can be expressed as;

$$\mathbf{265} = 2 \times 10^{-1} + 6 \times 10^{-2} + 5 \times 10^{-3} = 0.2 + 0.06 + 0.005 = 0.265$$

Binary number system

First convert 10 into its binary equivalent.

10.625



2^3	2^2	2^1	2^0
8	4	2	1
1	0	1	0

Convert the fractional part 0.625:

- **$0.625 \times 2 = 1.25$** → Keep 1 and continue with 0.25.
- **$0.25 \times 2 = 0.50$** → Keep 0 and continue with 0.5.
- **$0.5 \times 2 = 1.00$** → Keep 1 (stop as remainder is 0).

Combine the results: $10.625 = 1\ 0\ 1\ 0.1\ 0\ 1$ in binary.

Octal number system

The octal number system is a base-8 numeral system that uses eight digits: 0, 1, 2, 3, 4, 5, 6, 7.

Octal to Decimal Conversion

$(157)_8$



8^3	8^2	8^1	8^0
512	64	8	1
0	1	5	7

$$(157)_8 = (1 \times 8^2) + (5 \times 8^1) + (7 \times 8^0)$$

$$(157)_8 = (1 \times 64) + (5 \times 8) + (7 \times 1) = 64 + 40 + 7 = (111)_{10}$$

Thus, octal 157 is equivalent to decimal 111.

Octal number system

Decimal to Octal Conversion

- Convert the decimal number 111 to octal:
- Divide the decimal number by 8 and record the remainder:
 - $111 \div 8 = 13$ remainder **7**.
 - $13 \div 8 = 1$ remainder **5**.
 - $1 \div 8 = 0$ remainder **1**.
- Read the remainders from bottom to top: 157.
- Thus, decimal 111 is equivalent to octal 157.

Octal number system

Binary to Octal Conversion

Convert the binary number 1011101 to octal:

1. Group the binary digits into sets of three from right to left: 101 110 1.
2. Add leading zeros to complete groups of three: 000 101 110 001.
3. Replace each group with its octal equivalent:
 - Binary 101 = Octal 5
 - Binary 110 = Octal 6
 - Binary 001 = Octal 1
4. Combine the results: Octal 561.

Thus, binary 1011101 is equivalent to octal 561.

Hex number system

The **hexadecimal number system**, also known as the base-16 system, is widely used in computing and digital electronics.

It uses 16 symbols: the digits 0 to 9 represent values 0 to 9, and the letters A to F represent values 10 to 15

<u>Hexadecimal</u>	<u>Decimal</u>
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
A	10
B	11
C	12
D	13
E	14
F	15

Hex number system

Each position in a hexadecimal number represents a power of 16.

Starting from the right, the first position is 16^0 , the second is 16^1 , the third is 16^2 , and so on.

For example, the hexadecimal number **2F3** is computed as:

$$(2F3)_H = (2 \times 16^2) + (F \times 16^1) + (3 \times 16^0)$$

$$(2F3)_H = (2 \times 256) + (15 \times 16) + (3 \times 1) = 512 + 240 + 3 = \mathbf{755} \text{ (in decimal).}$$

Hex number system

Binary to Hexadecimal Conversion

Convert the binary number 110111101 to hexadecimal:

Group the binary digits into sets of 4: **0001 1011 1101**.

Convert each group to hexadecimal:

Binary 0001 = Hexadecimal 1

Binary 1011 = Hexadecimal B

Binary 1101 = Hexadecimal D

Combine the results: **1BD**.

Thus, binary 110111101 is equivalent to hexadecimal 1BD.

Hex number system

Hexadecimal to Binary Conversion

Convert the hexadecimal number 3A9 to binary:

Replace each hexadecimal digit with its 4-bit binary equivalent:

Hexadecimal 3 = Binary 0011

Hexadecimal A = Binary 1010

Hexadecimal 9 = Binary 1001

Combine the results: 0011 1010 1001.

Thus, hexadecimal 3A9 is equivalent to binary 1110101001.

Hex number system

Hexadecimal to Decimal Conversion

Convert hexadecimal 4F to decimal:

$$4F = (4 \times 16^1) + (F \times 16^0) = (4 \times 16) + (15 \times 1) = 64 + 15 = 79.$$

Thus, hexadecimal 4F is equivalent to decimal 79.

Decimal to Hexadecimal Conversion

Convert decimal 79 to hexadecimal:

1. Divide 79 by 16: $79 \div 16 = 4$ remainder **15**.
2. The quotient is 4 and the remainder is 15, which is F in hexadecimal.
3. Combine the results: 4F.

Thus, decimal 79 is equivalent to hexadecimal 4F.

Binary Coded Decimal (BCD)

BCD is a method of representing decimal numbers (0-9) in binary form, where each decimal digit is encoded as a separate 4-bit binary number.

In BCD, each decimal digit (0-9) is directly represented by a 4-bit binary equivalent:

- Decimal 0 = Binary 0000
- Decimal 1 = Binary 0001
- Decimal 2 = Binary 0010
- Decimal 3 = Binary 0011
- Decimal 4 = Binary 0100
- Decimal 5 = Binary 0101
- Decimal 6 = Binary 0110
- Decimal 7 = Binary 0111
- Decimal 8 = Binary 1000
- Decimal 9 = Binary 1001

Numbers beyond 9 are invalid in the BCD system because BCD only encodes individual decimal digits.

Binary Coded Decimal (BCD)

BCD Representation with Example

Suppose you want to represent the decimal number 245 in BCD:
Separate the decimal number into its individual digits: 2, 4, and 5.
Convert each digit into its 4-bit binary equivalent:

Decimal 2 = Binary 0010

Decimal 4 = Binary 0100

Decimal 5 = Binary 0101

Combine the binary equivalents: 245 in BCD = 0010 0100 0101.

In standard binary representation, the entire number is converted into binary:

Decimal 245 = $(2 \times 10^2) + (4 \times 10^1) + (5 \times 10^0)$

Convert it directly into binary: $(245)_{10} = (11110101)_2$

Binary Coded Decimal (BCD)

Addition in BCD

Add two decimal numbers, 5 and 6, in BCD:

Represent each number in BCD:

Decimal 5 = Binary 0101

Decimal 6 = Binary 0110

Perform binary addition: $0101 + 0110 = 1011$

The result 1011 is invalid in BCD (since it exceeds 9).

Apply correction by adding 6 (binary 0110): $1011 + 0110 = 00010001$.

The result is split into two valid BCD digits: **0001** (1) and **0001** (1), so the final answer is **11**.

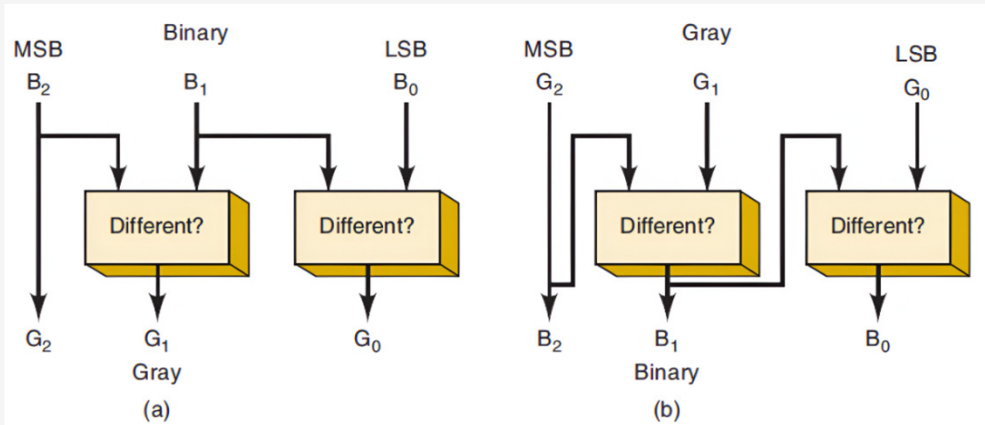
Gray Code

Gray Code (also called **Reflected Binary Code**) is a binary numeral system where two successive values differ in only one bit is also non-weighted codes.

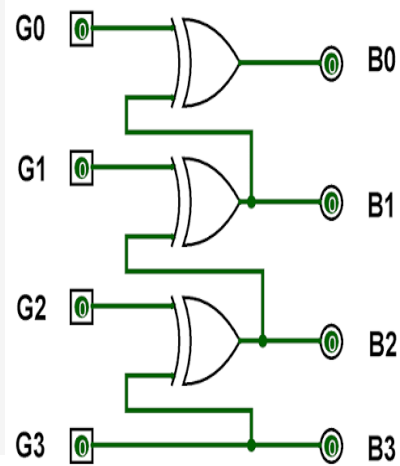
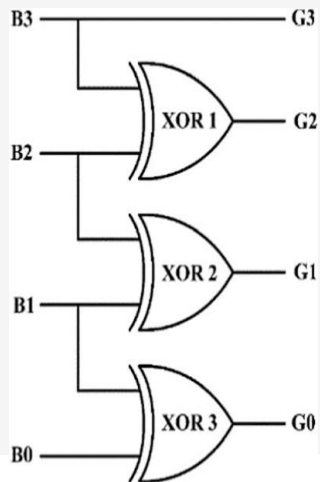
This unique property makes Gray Code particularly useful in minimizing errors in digital systems, especially in applications where smooth transitions between values are required.

- In **standard binary**, the transition from **0111 (7)** to **1000 (8)** requires **all four bits to change**.
- In **Gray Code**, only **one bit changes** between successive numbers.
 - Gray Code for **7** is **0100**.
 - Gray Code for **8** is **1100**.
 - Here, only the **most significant bit** changes (from 0 → 1).

Gray Code



B_2	B_1	B_0	G_2	G_1	G_0
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	1	0
1	0	1	1	1	1
1	1	0	1	0	1
1	1	1	1	0	0



Putting it together

Decimal	Binary	Hexadecimal	BCD	Gray
0	0	0	0000	0000
1	1	1	0001	0001
2	10	2	0010	0011
3	11	3	0011	0010
4	100	4	0100	0110
5	101	5	0101	0111
6	110	6	0110	0101
7	111	7	0111	0100
8	1000	8	1000	1100
9	1001	9	1001	1101
10	1010	A	0001 0000	1111
11	1011	B	0001 0001	1110
12	1100	C	0001 0010	1010
13	1101	D	0001 0011	1011
14	1110	E	0001 0100	1001
15	1111	F	0001 0101	1000

Floating-Point Numbers

Floating-point representation greatly increases the range of numbers, from the smallest to the largest, that can be represented using a given number of digits.

Floating-point numbers are in general expressed in the form;

$$\mathbf{N = m \times b^e}$$

- **m** is the fractional part, called the significand or mantissa.
- **e** is the integer part, called the exponent.
- **b** is the base of the number system or numeration.

Floating-Point Numbers

Decimal system

$$N = m \times 10^e$$

Hexadecimal system

$$N = m \times 16^e$$

Binary system

$$N = m \times 2^e$$

Decimal Floating-Point Representation

Decimal numbers are often represented in **scientific notation** in floating-point format.

The number 0.0003754 can be written as 3.754×10^{-4} , where 3.754 is the mantissa (significant digits) and -4 is the exponent indicating the position of the decimal point.

Similarly, the number 3754 can be written as 3.754×10^3 , where the decimal point is shifted three places to the left, and the exponent is $+3$.

Hexadecimal Floating-Point Representation

Example:

The hex number 257.ABF can be written as $2.57ABF \times 16^2$,
where 2.57ABF is the mantissa, and 2 is the base-16 exponent.

Binary Floating-Point Representation

Consider the mixed binary number $(110.1011)_2$

To normalize this, shift the radix point so that the most significant bit 1 is just after the point:

$$(110.1011)_2 = .1101011 \times 2^3$$

In floating-point notation, this can be written as $.1101011e+0011$, where:

Mantissa: $.1101011$

Exponent: $+3$ (binary 0011).

Binary Floating-Point Representation

Another example is $(0.000111)_2$:

Normalize by shifting the radix point to place the 11 after the point:

$$(0.000111)_2 = .111 \times 2^{-3}$$

In floating-point notation, this can be written as $.111e-0011$, where:

Mantissa: $.111$

Exponent: -3 (binary -0011).

For a negative number, such as $(-0.00000101)_2$:

$$\text{Normalize similarly: } (-0.00000101)_2 = -.101 \times 2^{-5}$$

In floating-point notation, this is $-.101e-0101$, where:

Mantissa: $-.101$

Exponent: -5 (binary -0101).